
実装設計書

スクリーニング用 スプレッドシート & GAS 設計書

Version 1.0

作成日: 2026年4月21日

対象: Googleスプレッドシート + Apps Script

発行: 株式会社Roadie

スクリーニング用Googleスプレッドシート & GAS 設計書

バージョン: 1.0

作成日: 2026年4月21日

関連DB: [イラストレーター情報DB \(Notion\)](#)

関連ドキュメント: [運用マニュアル \(Notion\)](#) / CLAUDE.md (illustrator-scraper)

1. 概要

目的

イラストレーター候補者のスクリーニング作業を、Notion を開くことなく Google スプレッドシート内で完結させる。スクリーナーは判定作業のみを行い、「○」判定されたレコードは GAS で自動的に Notion マスターDB へ転記される。

作業イメージ

Googleスプレッドシート「候補プール」シート

[Claude Codeが夜間バッチで候補を追加]

行1: 候補者A [○/保留/×] [S/A/B/C] [コメント]

行2: 候補者B [○/保留/×] [S/A/B/C] [コメント]

行3: 候補者C [○/保留/×] [S/A/B/C] [コメント]

...

[🔥 Notionへ転記ボタン]

ボタン押下

GAS (Google Apps Script)

1. ○判定かつ未転記のレコードを抽出

2. Notion マスターDBに新規ページ作成

ステータス=「候補」、画力ランク=スクリーナー値

3. スプレッドシート側を「転記済み」に更新

2. スプレッドシート構成

2.1 ファイル情報

項目	値
ファイル名	イラストレーター候補スクリーニング
オーナー	Takuya (Roadie)
共有設定	スクリーナー: 編集権限 / オーナー: 表示権限
サービスアカウント	編集者として共有必須 (illustrator-scraper 用)

2.2 シート構成

シート名	用途	主担当
候補プール	メインの作業シート	スクリーナー
転記ログ	Notion転記の実行履歴	GAS自動記録
設定	GAS の動作設定値	オーナー
README	スクリーナー向け簡易マニュアル	-

2.3 「候補プール」シートの列定義

列	ヘッダー	型	入力者	説明
A	検出日	DATE	自動	Claude Codeのバッチ実行日
B	検出元	TEXT	自動	どのシードアカウント経由か（例: <code>@example_artist</code> ）
C	Xアカウント	URL	自動	<code>https://x.com/username</code> 形式
D	表示名	TEXT	自動	X上の表示名
E	プロフィール	TEXT	自動	bio本文（改行込み、最大500文字）
F	フォロワー数	NUMBER	自動	
G	Pixivリンク	URL	自動	bio から抽出（なければ空）
H	ポートフォリオ	URL	自動	lit.link, Potofu 等
I	既存DB重複	TEXT	自動	<code>YES</code> / <code>NO</code> （Notion側で同じXアカウント存在確認）
J	判定	SELECT	スクリーナー	<code>○</code> / <code>保留</code> / <code>×</code> / （空）
K	画力ランク候補	SELECT	スクリーナー	<code>S</code> / <code>A</code> / <code>B</code> / <code>C</code> / （空）
L	コメント	TEXT	スクリーナー	自由記述
M	転記状態	TEXT	GAS	<code>未転記</code> / <code>転記済み</code> / <code>転記失敗</code>
N	転記日時	DATETIME	GAS	転記実行時の日時
O	NotionページID	TEXT	GAS	転記先のNotionページID（デバッグ用）

2.4 データ検証（プルダウン）

- J列（判定）：`○` , `保留` , `×`
- K列（画力ランク候補）：`S` , `A` , `B` , `C`
- M列（転記状態）：`未転記` , `転記済み` , `転記失敗`（初期値は `未転記`）

2.5 条件付き書式（推奨）

条件	書式
J列= ○	背景色：薄い緑
J列= ×	背景色：薄い灰色、文字：取り消し線
J列= 保留	背景色：薄い黄色
I列= YES（重複）	背景色：薄いオレンジ、注意喚起
M列= 転記済み	背景色：薄い青
M列= 転記失敗	背景色：薄い赤

2.6 固定・保護設定

- ・1行目（ヘッダー）：固定
- ・A～I列・M～O列: 「スクリーナー編集不可」で保護（GAS・Claude Code のみ書き込み）
- ・J～L列: スクリーナーが自由に編集可

3. 「転記ログ」シート

GAS が転記実行するたびに1行追記する。

列	ヘッダー	説明
A	実行日時	GAS実行時刻
B	実行者	実行したユーザーのメールアドレス
C	対象件数	○判定かつ未転記のレコード数
D	成功件数	Notionへ書き込めた件数
E	失敗件数	エラーになった件数
F	詳細ログ	失敗理由等

4. 「設定」シート

GAS が参照する設定値を管理。

セル	項目	値例	説明
A2	NotionAPIキー	secret_xxx...	Notion Integration の Secret
A3	NotionマスターDBデータソースID	1d5793bb-0629-4b6f-8a9a-3720c6a53139	データソース（コレクション）ID
A4	初期ステータス	候補	転記時に設定するマスターステータス
A5	見つけた人（デフォルト）	（スクリーナーメール→人名マッピング）	GAS内で別途変換
A6	バッチサイズ	50	一度の転記で処理する最大件数

5. GAS スクリプト

5.1 ファイル構成（スプレッドシートにバインドされたスクリプト）

```

スクリプト
├── main.gs      # エントリーポイント・メニュー定義
├── notion.gs    # Notion API 呼び出し
├── sheet.gs     # スプレッドシート操作
├── config.gs    # 設定シートからの値読み取り
└── utils.gs     # 共通ユーティリティ

```

5.2 メインコード: `main.gs`

```

/**
 * スプレッドシート起動時にカスタムメニューを追加
 */
function onOpen() {
  const ui = SpreadsheetApp.getUi();
  ui.createMenu('🔧 運用ツール')
    .addItem('📄 Notionへ転記', 'transferToNotion')
    .addSeparator()
    .addItem('🔍 Notion疎通テスト', 'testNotionConnection')
    .addItem('📋 転記ログを表示', 'showTransferLog')
    .addToUi();
}

/**
 * メインの転記処理
 */
function transferToNotion() {
  const ui = SpreadsheetApp.getUi();
  const sheet = SpreadsheetApp.getActiveSpreadsheet().getSheetByName('候補プール');
  const config = loadConfig();

  // 対象レコードを抽出
  const targets = findTransferTargets(sheet);

  if (targets.length === 0) {
    ui.alert('転記対象なし', '○判定かつ未転記のレコードがありません。', ui.ButtonSet.OK);
    return;
  }

  // 確認ダイアログ
  const response = ui.alert(
    '転記確認',
    `${targets.length}件のレコードをNotionへ転記します。よろしいですか？`,
    ui.ButtonSet.OK_CANCEL
  );

  if (response !== ui.Button.OK) return;

  // 転記実行
  const results = {
    success: 0,
    failure: 0,
    errors: []
  };

  targets.forEach(target => {
    try {
      const notionPageId = createNotionPage(target, config);
      markAsTransferred(sheet, target.rowIndex, notionPageId);
      results.success++;
    } catch (err) {
      markAsFailed(sheet, target.rowIndex, err.message);
      results.failure++;
      results.errors.push({
        row: target.rowIndex,

```



```

        xUsername: target.xUsername,
        error: err.message
    });
}

// レート制限対策 (Notion APIは3 req/sec)
Utilities.sleep(400);
});

// ログ記録
writeTransferLog(results, targets.length);

// 完了通知
ui.alert(
    '転記完了',
    `成功: ${results.success}件 / 失敗: ${results.failure}件\n\n詳細は「転記ログ」シートを確認してください。`,
    ui.ButtonSet.OK
);
}

/**
 * Notion 疎通テスト
 */
function testNotionConnection() {
    const ui = SpreadsheetApp.getUi();
    const config = loadConfig();

    try {
        const result = queryNotionDataSource(config.notionApiKey, config.dataSourceId, {
            page_size: 1
        });
        ui.alert(
            '疎通成功',
            `Notion APIへの接続に成功しました。DB内のレコード数（最新1件のみ取得）: ${result.results.length}`,
            ui.ButtonSet.OK
        );
    } catch (err) {
        ui.alert('疎通失敗', `エラー: ${err.message}`, ui.ButtonSet.OK);
    }
}

```

5.3 Notion API 呼び出し: `notion.gs`

```

/**
 * Notion マスターDB に新規ページを作成
 */
function createNotionPage(target, config) {
  const url = 'https://api.notion.com/v1/pages';

  const properties = {
    // タイトル（作家名） - 表示名を仮置き（後でオーナーが編集）
    '作家名': {
      title: [{ text: { content: target.displayName || target.xUsername } }]
    },

    // Xアカウント
    'Xリンク': {
      url: target.xUrl
    },

    // Pixivリンク
    ...(target.pixivUrl ? {
      'Pixivリンク': { url: target.pixivUrl }
    } : {}),

    // その他連絡先（ポートフォリオ）
    ...(target.portfolioUrl ? {
      'その他連絡先': { url: target.portfolioUrl }
    } : {}),

    // 検出元アカウント（新プロパティ）
    '検出元アカウント': {
      rich_text: [{ text: { content: target.detectedFrom || '' } }]
    },

    // 画力ランク候補 → 画力ランクにコピー（オーナーが後で確定）
    ...(target.powerRankCandidate ? {
      '画力ランク': {
        select: { name: target.powerRankCandidate }
      }
    } : {}),

    // マスターステータス
    'マスターステータス': {
      status: { name: config.initialStatus } // デフォルト: 候補
    },

    // 見つけた日
    '見つけた日': {
      date: { start: formatDate(new Date()) }
    },

    // 備考にスクリーナーコメントを記録
    ...(target.screenerComment ? {
      '備考': {
        rich_text: [{
          text: { content: `[スクリーナー] ${target.screenerComment}` }
        }]
      }
    } : {}),
  };

```

```

    }
  } : {}))
};

const payload = {
  parent: { data_source_id: config.dataSourceId },
  properties: properties
};

const options = {
  method: 'post',
  contentType: 'application/json',
  headers: {
    'Authorization': `Bearer ${config.notionApiKey}`,
    'Notion-Version': '2025-09-03' // 最新APIバージョンを使用
  },
  payload: JSON.stringify(payload),
  muteHttpExceptions: true
};

const response = UrlFetchApp.fetch(url, options);
const code = response.getResponseCode();
const body = JSON.parse(response.getContentText());

if (code !== 200 && code !== 201) {
  throw new Error(`Notion API エラー (${code}): ${body.message || '不明なエラー'}`);
}

return body.id;
}

/**
 * Notion データソースをクエリ（疎通確認用）
 */
function queryNotionDataSource(apiKey, dataSourceId, options = {}) {
  const url = `https://api.notion.com/v1/data_sources/${dataSourceId}/query`;

  const fetchOptions = {
    method: 'post',
    contentType: 'application/json',
    headers: {
      'Authorization': `Bearer ${apiKey}`,
      'Notion-Version': '2025-09-03'
    },
    payload: JSON.stringify(options),
    muteHttpExceptions: true
  };

  const response = UrlFetchApp.fetch(url, fetchOptions);
  const code = response.getResponseCode();
  const body = JSON.parse(response.getContentText());

  if (code !== 200) {
    throw new Error(`Notion API エラー (${code}): ${body.message || '不明なエラー'}`);
  }
}

```

```
return body;  
}
```

5.4 スプレッドシート操作: [sheet.gs](#)

```

/**
 * 転記対象レコードを抽出
 * 条件: J列=○ かつ M列=未転記
 */
function findTransferTargets(sheet) {
  const lastRow = sheet.getLastRow();
  if (lastRow < 2) return [];

  const data = sheet.getRange(2, 1, lastRow - 1, 15).getValues();
  const targets = [];

  data.forEach((row, idx) => {
    const rowIndex = idx + 2; // シート上の行番号 (1-indexed, ヘッダー考慮)
    const judgment = row[9]; // J列
    const status = row[12]; // M列

    if (judgment === '○' && status === '未転記') {
      targets.push({
        rowIndex: rowIndex,
        detectedDate: row[0],
        detectedFrom: row[1],
        xUrl: row[2],
        displayName: row[3],
        profile: row[4],
        followerCount: row[5],
        pixivUrl: row[6],
        portfolioUrl: row[7],
        isDuplicate: row[8] === 'YES',
        xUsername: extractUsername(row[2]),
        powerRankCandidate: row[10], // K列
        screenerComment: row[11] // L列
      });
    }
  });

  return targets;
}

/**
 * 転記済みマークを付ける
 */
function markAsTransferred(sheet, rowIndex, notionPageId) {
  sheet.getRange(rowIndex, 13).setValue('転記済み'); // M列
  sheet.getRange(rowIndex, 14).setValue(new Date()); // N列
  sheet.getRange(rowIndex, 15).setValue(notionPageId); // O列
}

/**
 * 転記失敗マークを付ける
 */
function markAsFailed(sheet, rowIndex, errorMessage) {
  sheet.getRange(rowIndex, 13).setValue('転記失敗');
  sheet.getRange(rowIndex, 12).setValue(
    (sheet.getRange(rowIndex, 12).getValue() || '') + ` [ERR: ${errorMessage}] `
  );
}

```

```

}

/**
 * X URLから username を抽出
 * 例: https://x.com/example → example
 */
function extractUsername(xUrl) {
  const match = String(xUrl).match(/(?:twitter|x)\.com\/(?:^\/\?)+/);
  return match ? match[1] : '';
}

/**
 * 転記ログを記録
 */
function writeTransferLog(results, targetCount) {
  const ss = SpreadsheetApp.getActiveSpreadsheet();
  const logSheet = ss.getSheetByName('転記ログ') || ss.insertSheet('転記ログ');

  // ヘッダーがなければ追加
  if (logSheet.getLastRow() === 0) {
    logSheet.getRange(1, 1, 1, 6).setValues([[
      '実行日時', '実行者', '対象件数', '成功件数', '失敗件数', '詳細ログ'
    ]]);
  }

  const errorDetail = results.errors
    .map(e => `行${e.row}(@${e.username}): ${e.error}`)
    .join('\n');

  logSheet.appendRow([
    new Date(),
    Session.getActiveUser().getEmail(),
    targetCount,
    results.success,
    results.failure,
    errorDetail
  ]);
}

```


5.5 設定読み込み: `config.gs`

```
/**
 * 設定シートから値を読み込み
 */
function loadConfig() {
  const sheet = SpreadsheetApp.getActiveSpreadsheet().getSheetByName('設定');
  if (!sheet) {
    throw new Error('「設定」シートが存在しません。');
  }

  // PropertiesServiceでAPIキーを管理する方式を推奨（より安全）
  const scriptProperties = PropertiesService.getScriptProperties();

  return {
    notionApiKey: scriptProperties.getProperty('NOTION_API_KEY')
      || sheet.getRange('A2').getValue(),
    dataSourceId: sheet.getRange('A3').getValue(),
    initialStatus: sheet.getRange('A4').getValue() || '候補',
    batchSize: sheet.getRange('A6').getValue() || 50
  };
}
```

5.6 ユーティリティ: `utils.gs`

```
/**
 * 日付をNotion API用にフォーマット (YYYY-MM-DD)
 */
function formatDate(date) {
  const y = date.getFullYear();
  const m = String(date.getMonth() + 1).padStart(2, '0');
  const d = String(date.getDate()).padStart(2, '0');
  return `${y}-${m}-${d}`;
}

/**
 * 転記ログシートを開く
 */
function showTransferLog() {
  const sheet = SpreadsheetApp.getActiveSpreadsheet().getSheetByName('転記ログ');
  if (sheet) {
    SpreadsheetApp.setActiveSheet(sheet);
  } else {
    SpreadsheetApp.getUi().alert('「転記ログ」シートがまだありません。');
  }
}
```

6. 初回セットアップ手順

6.1 スプレッドシート作成

1. 新規Googleスプレッドシートを作成
2. ファイル名を **イラストレーター候補スクリーニング** に変更
3. 4つのシートを作成: **候補プール**, **転記ログ**, **設定**, **README**
4. 「候補プール」シートに列ヘッダー (A~O列) を入力
5. J・K列にデータ検証でプルダウンを設定
6. 条件付き書式を設定 (§ 2.5参照)

6.2 Notion Integration 作成

1. <https://www.notion.so/profile/integrations> にアクセス
2. 「New integration」→ 内部インテグレーションとして作成
3. Secret をコピー
4. Notionのイラストレーター情報DBページで「...」→「接続の追加」→ 作成したインテグレーションを選択

6.3 Google Apps Script 設定

1. スプレッドシートメニュー: 「拡張機能」→「Apps Script」
2. 5つのファイルを作成 (main.gs, notion.gs, sheet.gs, config.gs, utils.gs)
3. 上記のコードをコピペ
4. 保存してからメニュー「プロジェクトの設定」→「スクリプトプロパティ」で追加:
 - Property: **NOTION_API_KEY**
 - Value: Notion Integration の Secret
5. 設定シート A3 に **NotionマスターDBのデータソースID** を入力
 - 値: **1d5793bb-0629-4b6f-8a9a-3720c6a53139**

6.4 サービスアカウント共有

1. illustrator-scraper の GCP サービスアカウントメールをスプレッドシートに「編集者」として共有
2. Claude Code バッチがこのスプレッドシートに書き込めることを確認

6.5 動作確認

1. 「候補プール」シートに手動で1行ダミーデータを入力
2. J列に  を入れる
3. メニュー「 運用ツール」→「 Notion疎通テスト」で接続確認
4. 「 Notionへ転記」を実行
5. Notionのイラストレーター情報DBにレコードが作成されることを確認

7. 運用ルール

7.1 スクリーナーのルール

- ・週1回、50件を目安に判定
- ・判定後、必ず「🔥 Notionへ転記」ボタンを押す
- ・迷った場合は「保留」を選択（オーナー判断に委ねる）
- ・スプレッドシートのA～I列・M～O列は絶対に編集しない

7.2 オーナーのルール

- ・毎週、スプレッドシートの「保留」一覧を確認し、追加判定する
- ・GAS エラーが発生したレコードは手動で対応

7.3 メンテナンス

- ・四半期に1回: 転記ログをチェックし、異常傾向がないか確認
- ・古い「転記済み」レコード（3ヶ月以上経過）は別シートにアーカイブ
- ・Notion Integration の Secret を半年に1回ローテーション

8. トラブルシューティング

症状	対処
ボタンメニューが表示されない	スプレッドシートを再読み込み。onOpen が呼ばれないと出ない
「転記失敗」が多発	Notion Integration の接続が切れている可能性。再度接続を追加
「Notion API エラー (401)」	APIキーが無効。スクリプトプロパティを確認
「Notion API エラー (404)」	データソースIDが誤っている、またはインテグレーションがDBに接続されていない
同じレコードが重複転記される	M列の「転記済み」判定が失敗している。手動で「転記済み」に修正

9. 改訂履歴

バージョン	日付	内容	担当
1.0	2026-04-21	初版作成	三村